THUNDER

Test your post before your audience does.

FINAL SUBMISSION KIT

Pitch deck · portal copy · 2-minute video · finalist pitch · demo · Q&A · checklist

github.com/harshjoshi23/Thunder

Prepared for Harshvardhan Joshi · Team Thunder

Urgent security action
An OpenAI key was exposed in chat. Rotate it before submission, replace it only in local/Render environment variables, and verify that no secret appears in Git history, screenshots, logs, or README.

1. Submission map

The portal requires public links. The safest format is one public URL per artifact, tested in an incognito window before submission.

Artifact	Recommended hosting	Final URL	Status
2-minute video	YouTube Unlisted or public Loom	[ADD PUBLIC VIDEO URL]	**Required**
Source code	Public GitHub repository	https://github.com/harshjoshi23/Thunder/tree/main	**Required**
Pitch deck	Google Slides/Drive “Anyone with link can view” or public GitHub PDF	[ADD PUBLIC DECK URL]	**Required**
Working demo	Render Web Service URL	[ADD PUBLIC DEMO URL]	**Optional, strongly encouraged**
Generated samples	Public PDF in repo or Drive	[ADD PUBLIC SAMPLE OUTPUT URL]	**Recommended**

Recommended public filenames

- Thunder_Pitch_Deck.pdf
- Thunder_Sample_Output.pdf
- Thunder_2_Minute_Demo.mp4
- Thunder_Submission_Screenshots.zip

> **Public access test**
> Open every link in a private/incognito window while logged out. A judge should not be asked to request access, sign in, clone the project, or wait for permission.

Final portal order

1. Project name and one-line summary
2. Video URL
3. GitHub URL
4. Pitch deck URL
5. Working demo URL
6. Sample output URL
7. Final incognito check
8. Submit and save a confirmation screenshot

2. Paste-ready portal copy

Project name

> **Thunder**
> Test your post before your audience does.

One-sentence description

Thunder turns historical comments into an evidence-backed audience twin, runs three independent AI jurors against a draft, and rewrites it before publication.

Short description

Creators currently learn whether a post works only after publishing. Thunder adds a pre-publication test: it transforms real historical comments into three evidence-backed audience segments, simulates how each segment interprets a new draft, challenges unsupported claims, calculates transparent diagnostics in TypeScript, and produces an improved five-slide carousel with traceable changes.

Detailed description

Thunder is a pre-publication audience intelligence and scenario-testing tool for LinkedIn and Instagram creators. A creator imports historical comments, adds context, and submits a draft. A LangGraph workflow builds exactly three audience segments, validates every cited comment ID, runs three independent juror agents in parallel, and sends their disagreement to an adversarial critic. The application then computes audience fit, clarity, save potential, discussion potential, and misinterpretation risk using visible deterministic formulas. A strategy agent resolves the trade-offs and produces a five-slide carousel, caption, CTA, and explanation of each change. Optional integrations generate a cover, create narration, ground claims against a source URL, and send the approved package into n8n. Thunder does not claim perfect human prediction or guaranteed virality; it provides a grounded rehearsal based on the creator's own audience data.

Target user

Comment-rich LinkedIn and Instagram educators, founders, and career/productivity creators with roughly 10k-100k followers who publish repeatedly but do not have a dedicated audience analyst.

Why it is different

- Evidence-backed: every segment and major recommendation links to original comment IDs.
- Adversarial: three audience jurors can disagree, and a critic challenges claims and assumptions.
- Transparent: final scores are calculated in TypeScript rather than invented by the model.
- Actionable: the output is not just feedback; it is a complete five-slide carousel and approved workflow package.

Technical implementation

Next.js and TypeScript frontend/backend, LangGraph orchestration, structured model outputs validated with Zod, three parallel audience jurors, deterministic evidence validation and scoring, Playwright/Vitest verification, optional Firecrawl source grounding, fal.ai cover generation, ElevenLabs narration, n8n workflow export, and Render deployment.

Repository

https://github.com/harshjoshi23/Thunder/tree/main

3. Two-minute submission video

Record the product, not the code. Keep the cursor slow, use a clean browser window, and speak continuously. Target 1:50-1:58 so upload processing does not reveal an overrun.

Time	Screen action	Voiceover
0:00-0:12	Open Thunder on the input screen.	Creators only learn whether a post works after they publish. Generic AI can write more content, but it does not understand how my own audience will interpret it.
0:12-0:25	Show historical comments, creator context, and the weak draft.	Thunder adds a pre-publication test. I give it historical comments, creator context, and the draft I am considering.
0:25-0:42	Run the test; show agent progress.	A LangGraph workflow builds an audience twin, validates evidence, and runs three independent audience jurors in parallel.
0:42-0:59	Open the three Audience Twin cards and evidence.	These are not generic personas. Each segment is backed by the exact comments that created it: beginners, busy professionals, and hustle skeptics.
0:59-1:16	Show Audience Jury disagreement.	The jurors disagree on purpose. Beginners feel shamed, busy professionals want a realistic system, and skeptics challenge the absolute claim.
1:16-1:32	Show diagnostics and guardrails.	Thunder flags exaggeration and missing context, then calculates transparent scores in TypeScript instead of asking the model to invent a virality number.
1:32-1:47	Show optimized hook and five slides.	It resolves those trade-offs into a stronger hook and a complete five-slide carousel, with every important change linked back to audience evidence.
1:47-1:56	Show before/after and optional integrations.	The before-and-after view makes the improvement visible, while fal.ai, ElevenLabs, Firecrawl, and n8n extend the creator workflow.
1:56-2:00	Return to the Thunder wordmark.	Thunder: test your post before your audience does.

Recording note

Do not spend the video explaining setup, API keys, fallback modes, or the README. Show the working creator journey and one clear technical proof point: evidence IDs plus deterministic scoring.

4. Three-minute finalist pitch

Use the five-slide deck. This script is intentionally compact so you can pause for emphasis without rushing.

Slide 1 - Problem (0:00-0:35)

Creators have a strange problem: they own years of audience insight inside their comments, but they still publish blind. Today the workflow is draft, publish, wait, and learn publicly. For a LinkedIn or Instagram educator, one misunderstood post can cost trust and create hours of recovery. Analytics arrive too late, and generic AI writes for an audience - not your audience.

Slide 2 - Solution (0:35-1:15)

Thunder is an audience pre-flight lab. It converts real historical comments into a reusable audience twin, tests a draft across three independent AI jurors, challenges the result, and rewrites the post before publication. The difference is three things: it is grounded because every segment cites real comments; adversarial because the jurors disagree and a critic pushes back; and measurable because the final scores are calculated in TypeScript.

Slide 3 - Output (1:15-2:00)

Here is the seeded example. The original hook says: post every day or you will never grow. Beginners feel shamed, busy professionals ask for a 20-minute system, and hustle skeptics challenge the absolute claim. Thunder rewrites it to: you do not need to post every day; you need a system that survives a full-time job. It produces five practical slides and improves audience fit, clarity, and save potential while reducing misinterpretation risk.

Slide 4 - Go-to-market (2:00-2:35)

The first users are comment-rich LinkedIn and Instagram educators with 10k to 100k followers. I would start with 20 design partners, run three pre-flight tests per creator each week, track which edits they accept, and compare published outcomes against each creator's own baseline. The product can start free, move to a $29 creator plan, and expand to a studio workflow for agencies.

Slide 5 - Why now / why me (2:35-3:00)

AI has made content abundant, so audience fit and trust are now the bottleneck. I am a solo builder completing a Master's in Data Science with software, automation, and AI engineering experience. Thunder is already a working vertical slice with multi-agent orchestration, evidence validation, deterministic scoring, tests, and creator workflow integrations. I would pilot it with real creators and keep building if repeated use improves confidence and outcomes.

5. Two-minute live demo

Time	Action
0:00-0:10	Open Thunder and say: “I am testing a productivity carousel before publishing.”
0:10-0:20	Load the seeded demo. Briefly point to the 15 historical comments, creator context, and weak absolute draft.
0:20-0:32	Click Run Audience Test. Let the agent-stage animation begin. Do not narrate every node.
0:32-0:52	Audience Twin: open one evidence drawer and show that C01/C02 map to real comments.
0:52-1:12	Audience Jury: compare two conflicting reactions and read the conflict summary.
1:12-1:29	Diagnostics: show one visible formula and the high misinterpretation-risk guardrail.
1:29-1:47	Optimized Carousel: read the improved hook and sweep across the five slide previews.
1:47-1:57	Before/After: show the score deltas; explicitly say lower risk is better.
1:57-2:00	Close: “Thunder tests interpretation before publication.”

Demo safety

- Preload the app and run one live analysis before the presentation.
- Keep a labeled seeded demo ready in a second tab.
- Do not trigger cover/video generation during the core demo unless it has already been tested.
- Open the Render app 10-15 minutes before judging to avoid a cold start.
- Use a stable browser zoom and never show environment variables or developer tools.

6. Judge Q&A

Is this just ChatGPT with cards?

A generic chat gives one unstructured opinion. Thunder builds a reusable audience model from the creator's own comments, preserves evidence links, runs the same draft through three independent jurors, applies deterministic scoring, and produces a complete workflow output.

Are these real people?

No. They are grounded scenario simulations built from patterns in supplied comments. Thunder is explicit about that limitation and does not claim perfect human prediction.

How do you prevent fabricated evidence?

Comments receive stable IDs. Every model-generated evidence ID is validated in TypeScript; nonexistent IDs are repaired, rejected, or removed before anything is shown.

Where do the scores come from?

The model supplies constrained factor ratings. TypeScript combines them with visible weighted formulas. The final 0-100 score is not invented directly by the model.

Why use multiple agents?

The tasks are genuinely different: audience discovery, independent segment reactions, adversarial criticism, and content strategy. Separating them creates disagreement and an auditable workflow instead of one agreeable answer.

Why n8n?

n8n is not the model. It receives the approved content package and continues the creator workflow - for example, human approval, storage, team notification, or scheduling.

What do fal.ai, ElevenLabs, and Firecrawl add?

fal.ai creates the carousel cover or media asset, ElevenLabs creates narration for repurposing, and Firecrawl provides optional source grounding for factual claims. The core audience test still works independently.

How would you validate that it works?

Start with 20 creators, measure repeated use and accepted edits, then compare post-level saves, comments, and negative feedback against each creator's own baseline.

What is the business model?

A free trial creates habit, a $29 creator plan supports repeated tests and history, and a higher studio plan adds multiple brands and approval workflows.

What would you build next?

Creator-specific persistent audience twins, post-performance feedback loops, team collaboration, platform connectors, and stronger evaluation against real published outcomes.

7. Final submission checklist

Code and security

[ ] Rotate the exposed OpenAI key.
[ ] Confirm `.env*` is ignored and no secret exists in Git history.
[ ] Push Cursor's final changes to `main`.
[ ] Confirm the public repository opens while logged out.
[ ] Run lint, typecheck, unit/API tests, Playwright, and production build.
[ ] Confirm README contains no voucher codes, private instructions, or personal setup notes.

Demo

[ ] Deploy as a Render Web Service.
[ ] Add production environment variables in Render only.
[ ] Verify `/api/health` and the complete browser journey.
[ ] Test Live / Seeded Demo / Recovery labels.
[ ] Open the app before judging to avoid a cold start.

Pitch deck

[ ] Keep the deck at five slides maximum.
[ ] Upload `Thunder_Pitch_Deck.pdf` or the editable deck publicly.
[ ] Verify the deck link requires no sign-in.
[ ] Replace sample visuals with final Playwright screenshots only if they make the slide clearer.

Video

[ ] Record at 1920×1080.
[ ] Keep runtime below 2:00.
[ ] Hide tabs, bookmarks, notifications, keys, and developer tools.
[ ] Upload to YouTube Unlisted or public Loom.
[ ] Test playback while logged out.

Portal

[ ] Add GitHub URL.
[ ] Add video URL.
[ ] Add pitch deck URL.
[ ] Add working demo URL.
[ ] Add sample output URL.
[ ] Test every link in incognito.
[ ] Submit before the deadline and save proof of submission.

> **Final recommendation**
> Submit the public Render URL, the five-slide PDF deck, the two-minute video, the GitHub repository, and the sample-output PDF. Keep the editable PPTX and DOCX for last-minute changes, but use stable public-view links in the portal.